



NCRD's Sterling Institute of Management Studies
(NAAC Accredited A+ Grade)

Nerul, Navi Mumbai

2025-2027

GENERATIVE AI 10-Day Workshop

Project Documentation

On

AI Chatbot Integration using OpenRouter API

Under the Guidance

of

Mr. Navneet Sir

By

Name : Swaraj Kharpude

Roll No : 92

SYMCA Div:B

Title

Building an LLM-Powered AI Chatbot using Python and the Google Gemini API

Aim

To understand the fundamentals of Generative AI by developing and configuring a Python-based chatbot that interacts with Large Language Models (LLMs) via the Google Gemini API.

Objectives

- To explore Generative AI fundamentals and understand how LLMs generate text.
- To comprehend core concepts like tokens, embeddings, and context window management.
- To implement a working Python application capable of sending inference requests to the Google Gemini 3.5 Flash model through the Google GenAI SDK.
- To securely manage API credentials within a Google Colab environment.

Theory & Background Knowledge

Before analyzing the code, it is essential to understand the underlying Generative AI framework that powers the chatbot:

- **Generative AI & LLMs:** Generative AI refers to systems capable of generating new content (text, code, images). Large Language Models (LLMs) are a subset of this, specifically trained on vast datasets of text to understand and generate human-like language based on statistical probability.
- **Transformer Architecture (High-Level):** Modern LLMs rely on the Transformer architecture. Instead of reading text sequentially, Transformers use a mechanism called *self-attention* to weigh the importance of all words in a sentence simultaneously, allowing the model to capture deep semantic context.
- **Tokens and Tokenization:** LLMs do not read words; they read "tokens." Tokenization is the process of breaking down text into smaller sub-word units.
- **Context Window:** The context window is the maximum number of tokens an LLM can process in a single request (both the input prompt and the generated output combined). In our implementation, the selected Google Gemini 3.5 Flash model is accessed through Google Gemini, allowing the prompt and generated response to be handled through the same API workflow.
- **Parameters, Training, and Inference:** A model's "parameters" are the billions of adjustable weights it learned during its initial *Pre-training* phase. Our Python code executes the *Inference* phase—where the trained model processes our prompt and generates a response in real-time.
- **Google Gemini 3.5 Flash Model:** Rather than using the generic openrouter/free endpoint, this implementation directly selects the NVIDIA Gemini 3.5 Flash 550B A55B free model

through the Google GenAI SDK. The exact model identifier used in the API request is gemini-3.5-flash.

Overview

This module demonstrates how to implement an AI chatbot within a Google Colab environment. Rather than binding directly to a single provider, this implementation leverages the official Google GenAI Python SDK and redirects it to the **Google Gemini API**. Google Gemini provides direct access to Gemini models through the Google GenAI SDK, allowing the Python application to send prompts and receive generated responses.

Prerequisites & Environment Setup

To execute this script successfully, the environment must be correctly configured to handle secure credentials.

Required Accounts

- **Google Account:** To execute the notebook via Google Colab.
- **Google Gemini Account:** To generate and use a Google Gemini API key.

Secure Key Management

Hardcoding API keys is a security risk. This project utilizes Google Colab's native Secrets manager:

1. Navigate to the  **Secrets** panel on the left sidebar of your Colab interface.
2. Click **Add new secret**.
3. Name the secret exactly: Swaraj_Kharpude.
4. Paste your Google Gemini API key into the **Value** field.
5. Toggle the **Notebook access** switch to allow the runtime to read the key.

Code:

```
! pip install -U langchain-google-genai
! pip install -U langchain
!pip install -q -U google-genai langchain
```

```
from google import genai
from google.colab import userdata
```

```
key = userdata.get("Swaraj_Kharpude")
```

```
client = genai.Client(api_key=key)
```

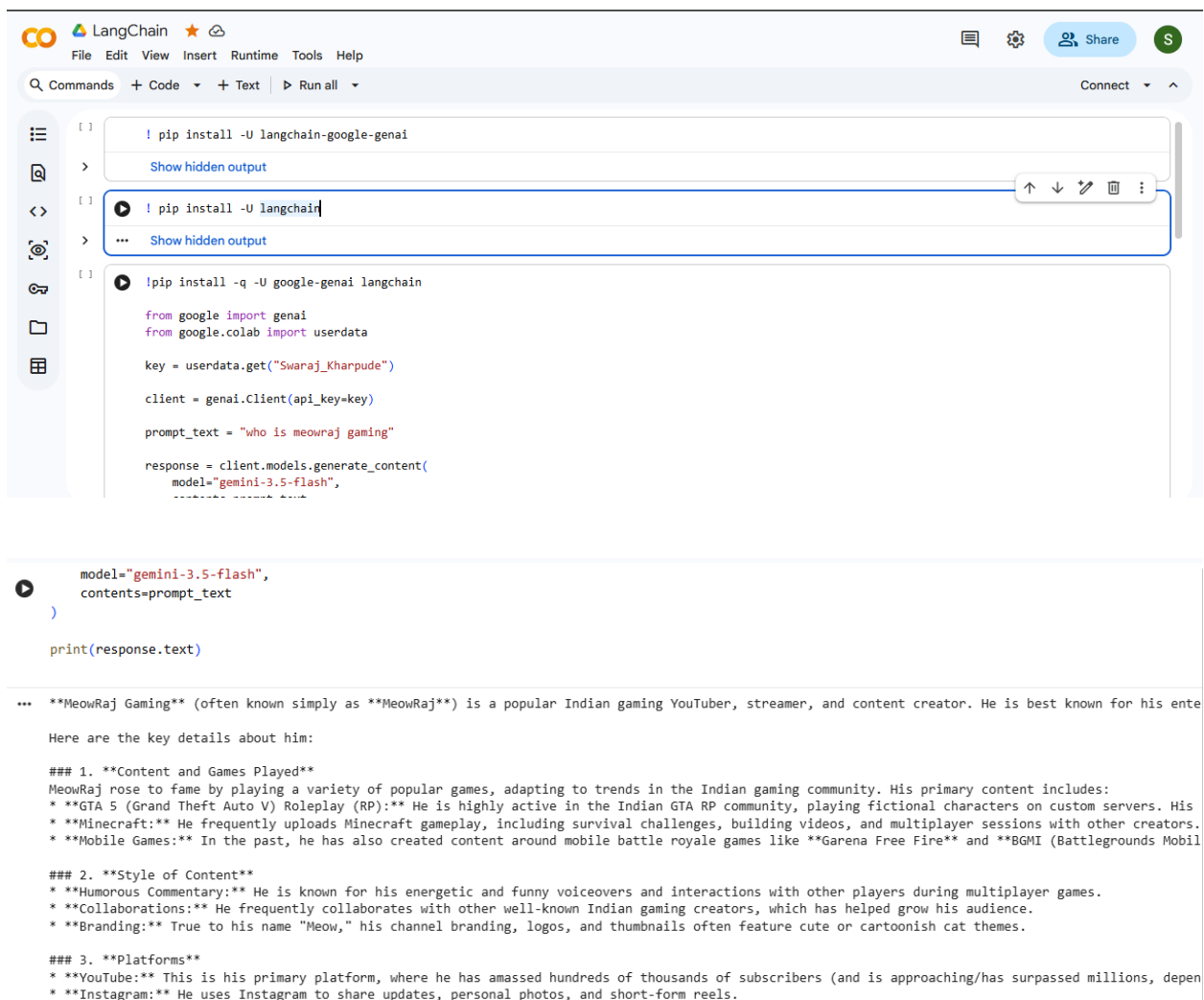
```
prompt_text = "who is meowraj gaming"
```

```
response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt_text
)
```

```
print(response.text)
```

Output :

After successful execution, the generated response for the prompt "who is meowraj gaming" from the Gemini 3.5 Flash model is printed in the Google Colab output cell.



```
! pip install -U langchain-google-genai

! pip install -U langchain

!pip install -q -U google-genai langchain

from google import genai
from google.colab import userdata

key = userdata.get("Swaraj_Kharpude")

client = genai.Client(api_key=key)

prompt_text = "who is meowraj gaming"

response = client.models.generate_content(
    model="gemini-3.5-flash",
    contents=prompt_text
)

print(response.text)
```

... **MeowRaj Gaming** (often known simply as **MeowRaj**) is a popular Indian gaming YouTuber, streamer, and content creator. He is best known for his ente

Here are the key details about him:

1. **Content and Games Played**

MeowRaj rose to fame by playing a variety of popular games, adapting to trends in the Indian gaming community. His primary content includes:

- GTA 5 (Grand Theft Auto V) Roleplay (RP):** He is highly active in the Indian GTA RP community, playing fictional characters on custom servers. His
- Minecraft:** He frequently uploads Minecraft gameplay, including survival challenges, building videos, and multiplayer sessions with other creators.
- Mobile Games:** In the past, he has also created content around mobile battle royale games like **Garena Free Fire** and **BGMI (Battlegrounds Mobil**

2. **Style of Content**

- Humorous Commentary:** He is known for his energetic and funny voiceovers and interactions with other players during multiplayer games.
- Collaborations:** He frequently collaborates with other well-known Indian gaming creators, which has helped grow his audience.
- Branding:** True to his name "Meow," his channel branding, logos, and thumbnails often feature cute or cartoonish cat themes.

3. **Platforms**

- YouTube:** This is his primary platform, where he has amassed hundreds of thousands of subscribers (and is approaching/has surpassed millions, depen
- Instagram:** He uses Instagram to share updates, personal photos, and short-form reels.

Conclusion & Key Takeaways

Building this AI chatbot bridges the gap between theoretical Generative AI concepts and practical application. By implementing this code, we successfully applied several core topics from the syllabus in a real-world scenario:

- **Generative AI Fundamentals & LLMs:** We moved from theory to practice by directly interacting with a Large Language Model to generate text dynamically based on a user prompt.
- **Parameters, Training, and Inference:** Through the Google Gemini API, we executed an active inference request. The model processed our input prompt (context) and generated a response, demonstrating the end result of model training and text generation in real time.
- **Google Gemini 3.5 Flash via Google GenAI SDK:** By explicitly using the `gemini-3.5-flash` model identifier, the application sends the prompt to the selected Gemini model through `client.models.generate_content()` and receives the generated response in `response.text`.
- **Python for GenAI Applications:** We utilized Python as the primary application layer to securely manage credentials, structure the API payload using the standard `google-genai` library, and parse the complex JSON response.

Ultimately, this project serves as the foundational stepping stone. By successfully establishing an API connection and parsing an LLM's output, the groundwork is laid for advancing to more complex orchestration frameworks like **LangChain**, running **Local LLMs via Ollama**, and building full-scale AI agents.